Yoobee College of Creative Innovation
Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project

Individual Reflection Report

Leading a two-person quantum project when the quantum computer goes offline

Student	Anish Shrestha
Student ID	270889991
Role in the group	Team lead — planning, integration, Tasks 2 and 3, shared code, GitHub/Colab, Quokka testing, report editing
Group partner	Sandhya Tiwari (270918511)
Lecturer	Dr Helena Bahrami
Reflective model	Gibbs' reflective cycle (Gibbs, 1988)
Date	4 November 2026
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration

I, Anish Shrestha, declare that this reflection is my own individual work for MSE802 Assessment 2. It describes my own experience of the group project with Sandhya Tiwari. Sources are acknowledged in APA 7th style, and the project facts and numbers come from our submitted notebooks, results files and Git history.

Use of digital tools: a generative AI assistant was used for language editing and for drafting parts of the project code and documents; the experiences, judgements and plans in this reflection are my own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID	Signature / date
Anish Shrestha	270889991	

Table of Contents

Declaration..... 1
Table of Contents..... 2
Summary..... 3
1. Introduction..... 4
1.1 What is this reflection for?..... 4
1.2 Why did I choose Gibbs' cycle?..... 4
1.3 What was the project?..... 4
2. My Role and How We Divided the Work..... 5
2.1 What did I do as team lead?..... 5
2.2 How did the team develop over time?..... 5
3. Stage 1 – Description: What Happened?..... 7
3.1 Event 1: The quantum computer was not there..... 7
3.2 Event 2: Block C needed more than one gate..... 7
3.3 Event 3: The game ignored our clicks..... 7
3.4 Event 4: Colab could not find Cirq..... 8
4. Stage 2 – Feelings: What Was I Thinking and Feeling?..... 9
5. Stage 3 – Evaluation: What Was Good and What Was Bad?..... 10
5.1 How did the re-run on the real device turn out?..... 10
6. Stage 4 – Analysis: Why Did Things Happen This Way?..... 12
6.1 Why did the Quokka outage hurt less than it could have?..... 12
6.2 Why did I misread block C?..... 12
6.3 What did the redesigned circuit teach me about quantum computing?..... 12
6.4 Why did the environment problems take so long?..... 13
7. Stage 5 – Conclusion: What Else Could I Have Done?..... 14
8. Stage 6 – Action Plan: What Will I Do Next Time?..... 15
9. Final Thoughts..... 15
References..... 15
Appendix – Evidence Behind This Reflection..... 17

(If page numbers are missing, right-click the table and choose Update Field.)

Summary

This reflection looks back on my work as **team lead** of our MSE802 Assessment 2 group project. I use **Gibbs' reflective cycle** (Gibbs, 1988) because the project had a few clear, dramatic moments that suit its step-by-step structure: the Quokka quantum computer was offline when we needed it, our first answer to Task 2 turned out to be thinner than the lecturer expected, and the game in Task 3 ignored mouse clicks. I describe what happened, how I felt, what went well and badly, why it happened, what I conclude, and what I will do differently. The main lesson is simple: **a leader must test the assumptions that the whole plan depends on, and must do it early**. I assumed that the device would be online and that our reading of the brief was complete. Both assumptions failed, and I spent extra days recovering. On the positive side, the shared code, the labelled fallback and the Git history made every recovery fast and honest.

1. Introduction

1.1 What is this reflection for?

Reflection turns experience into learning. Schön (1983) separates *reflection-in-action*, the quick thinking we do while solving a problem, from *reflection-on-action*, the slower review we do afterwards. This document is reflection-on-action. It is written for my lecturer, but it is also a record for me: next time I lead a technical team, I want to remember which habits saved time and which habits cost time.

1.2 Why did I choose Gibbs' cycle?

Gibbs' model has six stages: description, feelings, evaluation, analysis, conclusion and action plan (Figure 1). I chose it for three reasons. First, it forces me to separate *what happened* from *how I felt about it*, which is hard when something goes wrong close to a deadline. Second, its evaluation and analysis stages push me to link my experience to theory instead of only telling a story. Third, the final action plan makes the reflection useful for the future. My partner used a different model for her own reflection, so our two documents look at the same project from different angles.

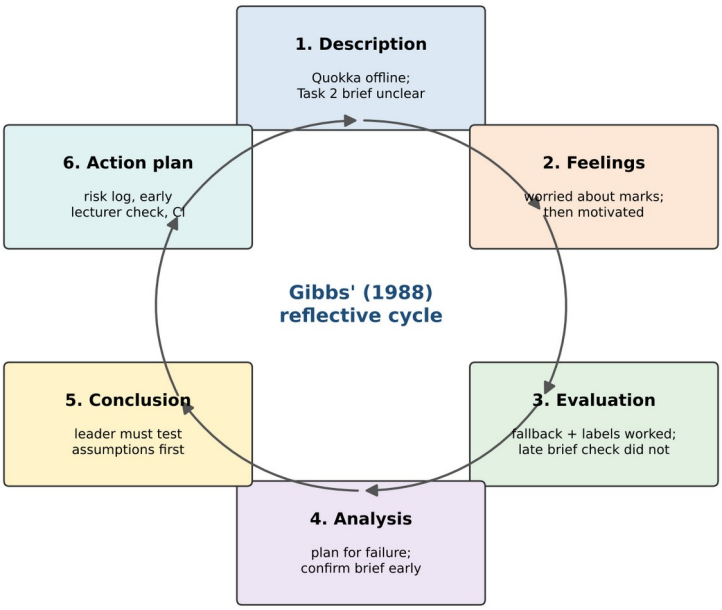


Figure 1. *Gibbs' reflective cycle as I applied it to this project*

1.3 What was the project?

Assessment 2 asked our group to build and test four pieces of quantum software: a Bell state in Cirq (Task 1), a given circuit plus our own circuit in Qiskit and QASM (Task 2), a completed quantum tic-tac-toe game with an analysis (Task 3), and an analysis of a small quantum machine-learning classifier (Task 4). Every circuit had to run on **Quokka**, the course's quantum computing emulator, and the results had to be explained. The lecturer later added that we must **compare our own computer with Quokka** for speed and outcome. Everything was delivered as notebooks, a group report, launch scripts for Windows and macOS, a GitHub repository and a Google Colab notebook.

2. My Role and How We Divided the Work

2.1 What did I do as team lead?

As team lead I took responsibility for the parts that hold the project together and for the larger technical tasks. Table 1 shows the split we agreed. I use the RACI idea: **R** = did the work, **A** = accountable for the final quality, **C** = consulted or reviewed, **I** = kept informed.

Table 1. *Who did what (R = responsible, A = accountable, C = consulted / reviewed, I = informed)*

Work package	Anish Shrestha	Sandhya Tiwari
Project plan, task order, meeting notes, deadlines	R, A	C
Shared helper `quokka_helper.py` (QASM cleaning, Quokka call, fallback, failover)	R, A	C
Task 1 – Bell state in Cirq	A, C	R
Task 2 – given circuit (block C) and custom QASM half adder	R, A	C
Task 3 – completing the tic-tac-toe code, bug fixes, played games	R, A	C
Task 3 – gate section and Word document	R, A	C
Task 4 – ML analysis, iteration monitoring, local versions	A, C	R
Runner, .bat / .command launchers, requirements, maintenance guide	R, A	I
GitHub repository, Colab notebook, ZIP package	R, A	I
Quokka diagnosis, multi-device testing, laptop vs Quokka comparison	R, A	C
Group report: structure, integration, final editing	R, A	C
Reference checking and proofreading	C	R

In practice, my working day as lead had two halves. In the first half I built or fixed shared pieces so that my partner was never blocked. For example, the helper had to be ready before she could send her Task 1 circuit to Quokka. In the second half I worked on my own tasks (Tasks 2 and 3) and integrated finished pieces into the report and the repository. I tried to follow a rule I had read about cooperative learning: a group works best when each member depends on the others' work *and* is individually accountable for a visible part of it (Johnson & Johnson, 2009). Giving Sandhya full ownership of Task 1 and Task 4 made her accountable for a visible result, while the shared helper created the positive dependence between us.

2.2 How did the team develop over time?

Tuckman (1965) describes four stages of small groups: forming, storming, norming and performing. Our group moved through them quickly. **Forming** was the first brainstorming session, where we read the brief together and listed the four tasks. **Storming** was short but real: we disagreed about whether to write separate reports per task or one combined report, and about how much theory to include. We agreed on one combined report with a WH-question style ("What…? Why…? How…?"), because it keeps each section focused on a question a marker might ask. **Norming** came when we agreed on fixed

conventions: 1000 shots for every circuit, every result saved as a JSON file with its source, and inline comments on every line of code. After that we were **performing**: most of the code and the first full report were committed on a single day, 31 October 2026 (Figure 2).

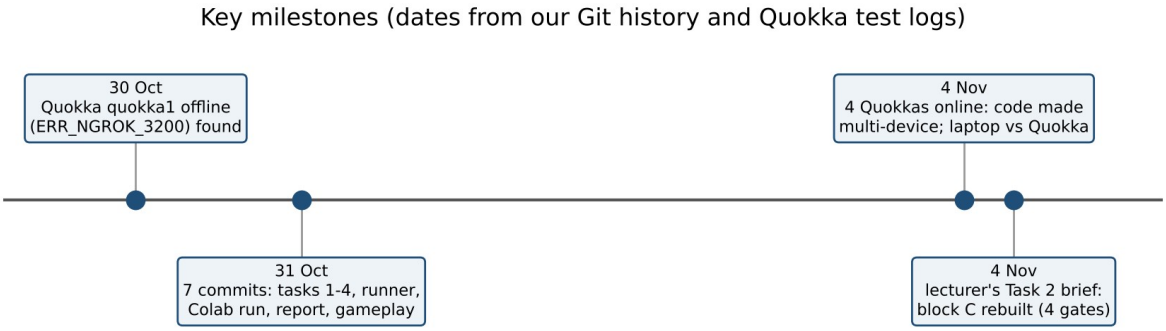


Figure 2. *Key milestones of the project, taken from our Git history and the Quokka test logs*

3. Stage 1 – Description: What Happened?

Four events shaped my experience more than anything else. I describe them briefly and factually here; the later stages examine them.

3.1 Event 1: The quantum computer was not there

On 30 October 2026, when the first circuits were ready, every request to the course address `quokka1.quokkacomputing.com` returned **HTTP 404 with the code ERR_NGROK_3200**. This code means that the public address still exists but no server is connected behind it. I tested from our workspace, from Google Colab and from a normal web browser, and I also tried quokka2, quokka3 and quokka4. All of them were offline at that time. I asked whether a login or API key was missing; after reading the official Quokka notebooks I found that no key is used at all. A background script re-checked the device every two minutes for two hours on 31 October, and it never came back. We therefore finished the first full version of the project on a **local simulator** and labelled every result as such.

On 4 November 2026 I checked again. This time I signed in to the Quokka website and found the official **Fleet Status** board, which lists eleven cloud devices, quokka1 to quokka11. Four of them (quokka2, quokka3, quokka4 and quokka5) answered with real results, while quokka1, the address in our course notebook, was still offline (Figure 3). I changed the shared code so it keeps a list of devices and moves to the next one if a device fails, and we re-ran the tasks on the real device.

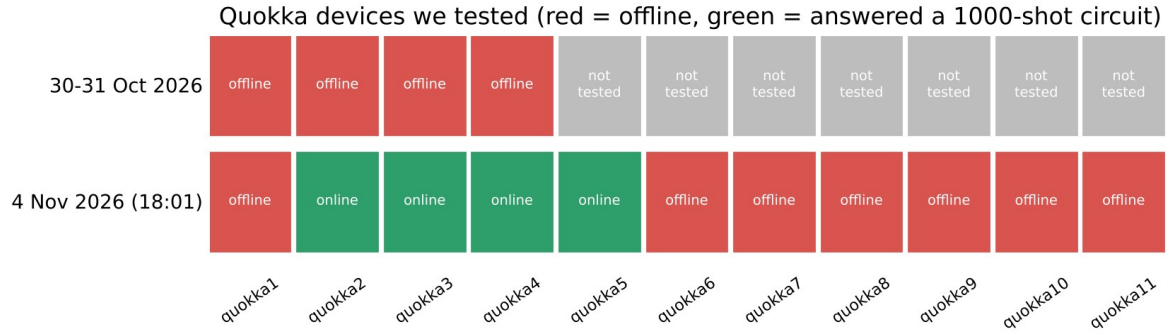


Figure 3. *Quokka devices I tested: all offline on 30–31 October; four of eleven online on 4 November*

3.2 Event 2: Block C needed more than one gate

Task 2 showed an H gate and an X gate entering a box called "C". In our first version I read the box as a single CNOT gate, and the lecturer had confirmed that C includes a CNOT. Later, when I asked for more detail, the lecturer explained that block C should contain **at least two or three gates of our choice, including a controlled gate**, and that we must explain whether entanglement is created and what the results mean. Our first answer (H, X, CNOT) was correct but too small. I redesigned block C as **CNOT → Z → CNOT → H**: the first CNOT makes a Bell pair, Z flips its phase, and the last two gates read the pair in the Bell basis.

3.3 Event 3: The game ignored our clicks

The tic-tac-toe game in Task 3 uses buttons (ipywidgets). In Google Colab it worked, but in a local Jupyter Notebook the buttons appeared and nothing happened after a click. The cause was that the button handler printed to a cell that had already finished; Jupyter quietly dropped the output. I fixed it by giving the game its own **Output widget**, writing all output inside it, and marking the change as `[CHANGED]` in the code. Then we could play three real games by clicking, and I kept screenshots as evidence.

3.4 Event 4: Colab could not find Cirq

The first full run in Google Colab failed with "No module named cirq". Colab installed the package into one Python, but the notebooks were executed by a different Jupyter kernel. I changed the runner so that it installs any missing package into its *own* Python and registers a matching kernel. The second Colab run finished all four tasks.

4. Stage 2 – Feelings: What Was I Thinking and Feeling?

When Quokka failed, my first feeling was **worry**. The rubric's top band talks about running circuits on the Quokka device, and as team lead I felt that a missing device was *my* problem to solve, even though it was outside our control. I remember checking the address many times in one evening, which was not productive. After about an hour the worry turned into **determination**: if I could not control the device, I could control how honestly and how quickly we would react when it came back. That thought led to the labelled fallback and the one-command re-run.

When I learned that block C needed more gates, I felt **embarrassed** for a moment, because I had treated a confirmation about one gate as a confirmation of the whole design. The embarrassment was useful: it showed me how easily a team lead can stop asking questions once the first answer sounds right. Very soon it became **curiosity**, because designing a meaningful four-gate block was more interesting than the one-gate version. Watching the full circuit return `11` in **1000 out of 1000 shots** on quokka2, and the control circuit without Z return `10` in 1000 out of 1000, was one of the most satisfying moments of the course.

The Task 3 click problem produced **frustration**, because the bug left no error message. Fixing it gave me **confidence**: I learned to suspect the environment (kernel, widget manager, output area) and not only my own code. Throughout the project I also felt a steady **responsibility towards my partner**. I did not want her to wait for me, so I sometimes worked late on shared pieces. That was good for her progress but not always good for my own rest.

5. Stage 3 – Evaluation: What Was Good and What Was Bad?

Table 2. *Evaluation of the main decisions I made as team lead*

Decision	What went well	What went badly
One shared helper for all tasks	One fix (e.g. Quokka's gate set, the device list) repaired every task at once	Sandhya depended on my code; when I changed it she had to re-run her notebooks
Labelled local fallback	Work never stopped; no simulator number was ever presented as device data	The first full report had to be rewritten when Quokka returned
Fixed seed for the simulator	Results became repeatable on every computer, including Colab	It hid real randomness until I explained it in the report
Reading block C as one CNOT	Correct physics and quick progress	Too small for the lecturer's intention; the redesign came late
Committing early and often to Git	Every change can be traced; the history became evidence for the report	Early commits mixed many changes; messages could be more specific
Automated checks (Playwright clicks, background re-checks)	Proof of gameplay and of the outage without manual screenshots	Took time to set up; one watcher ran for two hours with no result

Overall, the **technical** decisions worked well. The weakest part was the **requirements** decision: I relied on my own reading of a diagram for too long. A second weak point was workload balance. Figure 4 groups the problems we found and fixed. Most of them were in the Quokka/network and environment areas. These are the integration areas that I owned, so most of the fixing naturally fell on me.

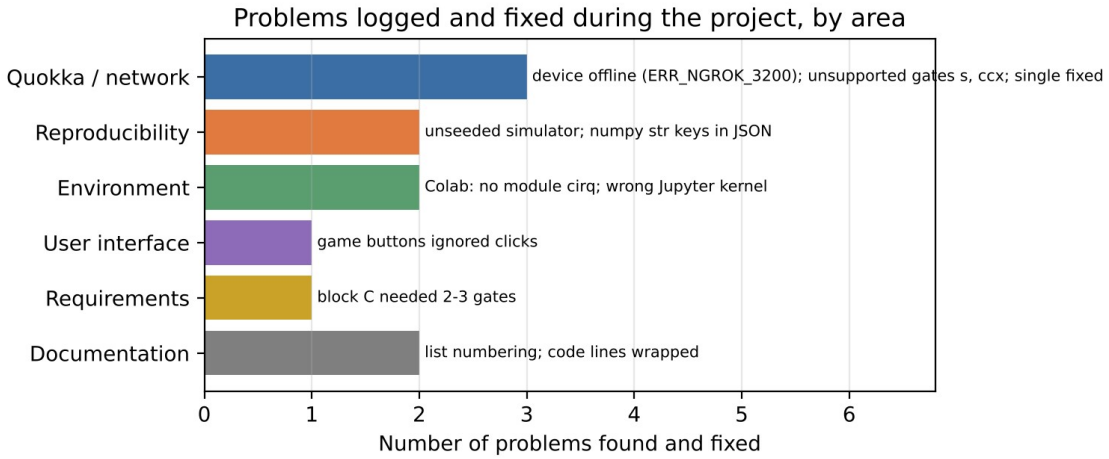


Figure 4. *Problems found and fixed during the project, grouped by area*

5.1 How did the re-run on the real device turn out?

When four Quokkas came back, I re-ran every task with one command. Task 1 gave **501 × 00 and 499 × 11**, and the redesigned Task 2 gave exactly the predicted deterministic outcomes. The half adder gave four correct rows (261, 242, 250, 247). The lecturer had also asked us to compare our own computer with Quokka. I wrote a small comparison tool that runs the same circuit on the project computer and on every online Quokka, five times each, with one untimed warm-up. Figure 5 shows the result: the project computer needed about **0.08 s** per 1000-shot run, and Quokka needed about **0.9–1.2 s**. However, the Quokka server itself reported only **2–4 ms** of work. Almost all of the difference is the trip

over the internet. The outcomes were equally accurate: the distance from theory (TVD) stayed below 0.015 for every target.

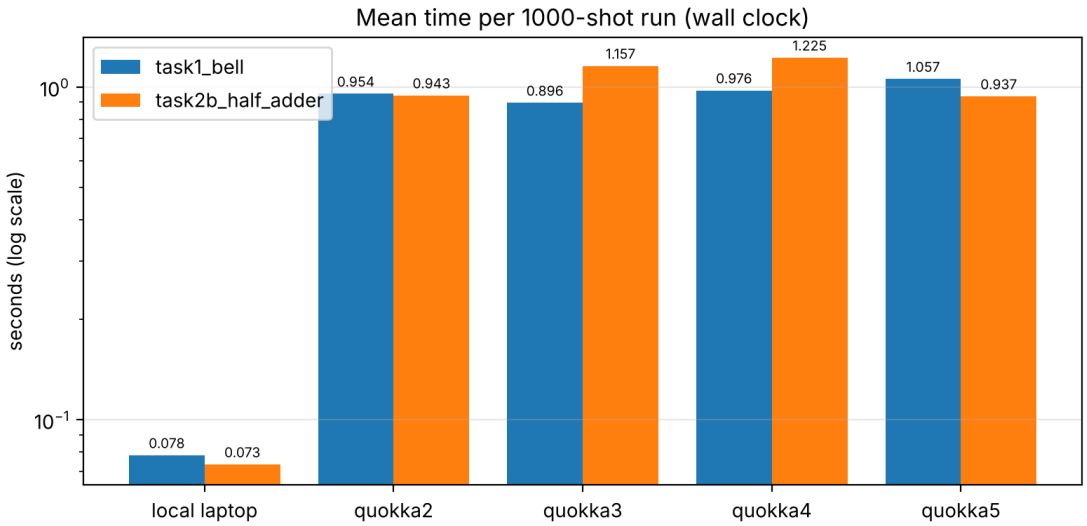


Figure 5. *Mean wall-clock time per 1000-shot run: project computer vs four Quokka devices (log scale)*

6. Stage 4 – Analysis: Why Did Things Happen This Way?

6.1 Why did the Quokka outage hurt less than it could have?

Boehm (1991) argues that software projects fail most often on risks nobody wrote down, and that each top risk needs a *contingency plan* prepared before it happens. Without planning it formally, we had one: the helper always had a local path. What I did *not* have was a written risk list. If I had written "device offline" on day one, I would have built the device list and the status check before the outage, and would have found quokka2 to quokka5 earlier. The lesson from Boehm is not only to have a fallback, but to *monitor* the risk. The fleet board I found on 4 November is exactly such a monitor.

There is also a scientific reason why the fallback was acceptable. Quokka emulates an *ideal* quantum computer, so its results should match a noiseless simulator up to random shot noise. Real NISQ hardware would add gate and readout errors (Preskill, 2018). This is why I insisted that every result carries its source label. A noiseless simulator and an ideal emulator should agree, but a reader must still be able to tell which one produced a number. Wilson et al. (2014) recommend the same kind of habit for scientific code: record provenance, automate repeatable steps and keep everything under version control.

6.2 Why did I misread block C?

I think there were two causes. The first is **confirmation bias**: when the lecturer confirmed that C was a CNOT, I heard "C is only a CNOT", because that matched my drawing. The second is my leadership style at that moment. Hersey and Blanchard's (1969) situational leadership model says a leader should adapt between *telling*, *selling*, *participating* and *delegating* to the situation. Under time pressure I was mostly *telling*: I decided the interpretation and moved on, instead of *participating*, which would have meant opening the question to my partner and the lecturer. The redesign shows the value of the other approach. Once we discussed what the circuit *should demonstrate*, the answer became richer. The circuit creates entanglement, changes it, and measures it.

6.3 What did the redesigned circuit teach me about quantum computing?

The new block C is a small version of *superdense coding* (Bennett & Wiesner, 1992). One local gate (Z) on one qubit moves the pair from $|\Psi^+\rangle$ to $|\Psi^-\rangle$, and a Bell measurement detects it with certainty. Before this project I knew from the lectures that "phase cannot be measured directly". Now I have seen it: the Z gate changes no probability in the checkpoint run (still only `01` and `10`, 532 and 468 on Quokka, about two standard deviations from 500, which chance produces in roughly one run in twenty), yet it flips the final answer from `10` to `11` when interference follows (Nielsen & Chuang, 2010). I also measured that the entanglement entropy goes from 0 to 1 bit after the first CNOT and back to 0 after the Bell measurement. This is a concrete number for an idea that used to be only words for me.

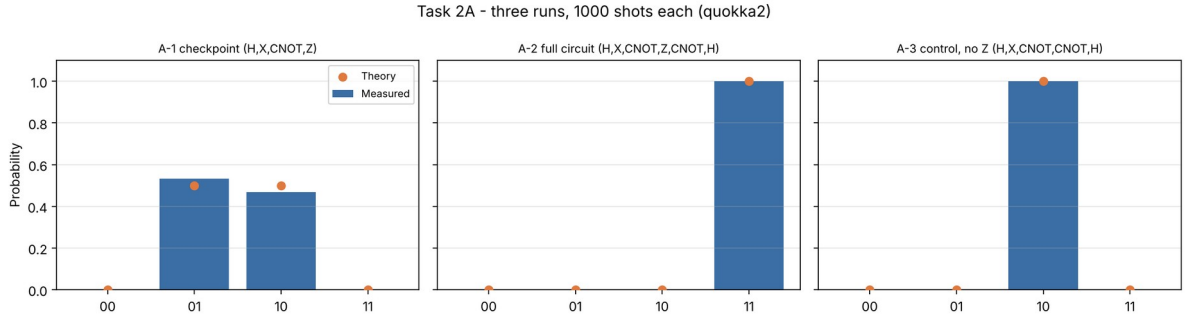


Figure 6. *My redesigned Task 2A on Quokka: checkpoint (entangled), full circuit (always 11) and control without Z (always 10)*

6.4 Why did the environment problems take so long?

Both the Colab error and the silent buttons came from the same blind spot. I tested my code in one environment and assumed it behaved the same everywhere. Notebooks hide the environment. Which Python, which kernel and which widget manager are in use is not visible in the code. Kolb's (1984) experiential learning cycle explains why these lessons stick: I had a concrete experience (a failure), observed it, formed a rule ("test in every target environment") and then actively tested that rule in the next run, which worked in Colab, Jupyter and VS Code.

7. Stage 5 – Conclusion: What Else Could I Have Done?

- **Confirm the brief with the lecturer before building**, especially when a diagram is open to interpretation. One short email at the start would have saved the Task 2 redesign.
- **Write a risk register on the first day.** The two biggest risks (device unavailable, unclear requirement) were predictable.
- **Check all available devices, not only the one in the notebook.** The working Quokkas were listed on the official website the whole time; I only found the board when I signed in.
- **Balance the workload more openly.** Because I owned the shared and integration work, I carried most of the late fixes. I should have asked my partner to review my integration work earlier, which would also have spread the knowledge.
- **Keep commits small.** Seven large commits tell the story, but smaller ones would make it easier to review.

I also want to keep several things. These are: the honest source label on every result, one runner command that reproduces everything, the habit of predicting results with mathematics *before* running a circuit, and the decision to give my partner clear ownership of whole tasks.

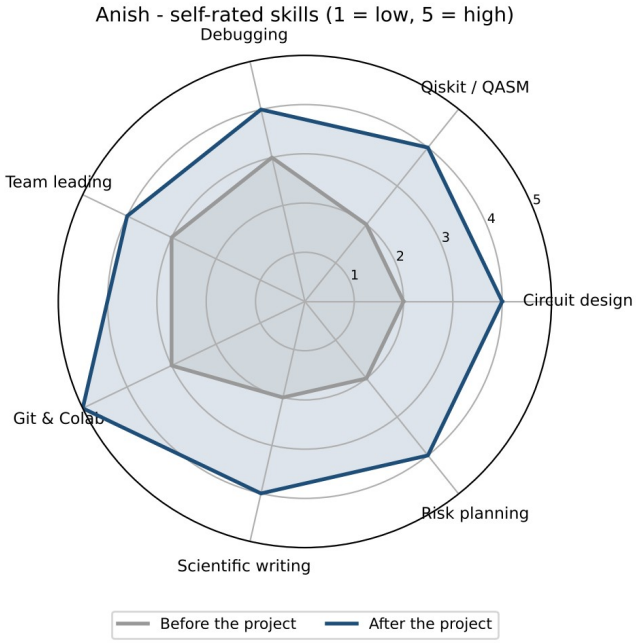


Figure 7. *My own rating of my skills before and after the project (1 = low, 5 = high)*

Figure 7 is my honest self-assessment. The largest gains were in **Git and Colab**, **circuit design** and **risk planning**. Risk planning moved from a weak point to a skill I now use on purpose. Team leading improved less, because leading a two-person team is a small step; I want to practise it in a larger group.

8. Stage 6 – Action Plan: What Will I Do Next Time?

Table 3. *My action plan (SMART: specific, measurable, achievable, relevant, time-bound)*

Goal	Action	How I will know it worked
Clear requirements	In the first week of any assignment, send the lecturer a one-page summary of how we read each task and ask for corrections	No redesign caused by a misread brief
Visible risks	Keep a five-line risk register in the repository README; review it at every meeting	Every major problem was on the list before it happened
Environment testing	Add an automated test that runs every notebook in a clean environment on each push (e.g. GitHub Actions)	A broken notebook is caught within minutes, not at submission
Fair workload	Track tasks on a shared board with owner and estimate; rotate integration duties	Both members' hours stay within about 20% of each other
Quantum depth	Rebuild the Task 2 idea as full superdense coding (all four Bell states) and run it on a noisy simulator	I can explain how noise changes the 1000/1000 result
Communication	Write a short summary of each meeting in the repository within 24 hours	My partner never needs to ask what was decided

I will start the first two actions in my next course assignment, because they cost almost nothing and address the two failures that cost the most time here.

9. Final Thoughts

Leading this project showed me that technical skill and leadership are linked. The most valuable code I wrote was not a clever circuit. It was the boring helper that made every failure recoverable and every result traceable. The most valuable leadership lesson was to keep asking questions after the first answer sounds right. The quantum computer being offline felt like bad luck at the time. Looking back, it was the event that taught our team the most about honest, reproducible engineering.

References

Bahrami, H. (2026). *Quokka access* [Unpublished Jupyter notebook, quokka_access.ipynb, Week 4]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bennett, C. H., & Wiesner, S. J. (1992). Communication via one- and two-particle operators on Einstein-Podolsky-Rosen states. *Physical Review Letters*, *69*(20), 2881–2884. https://doi.org/10.1103/PhysRevLett.69.2881

Boehm, B. W. (1991). Software risk management: Principles and practices. *IEEE Software*, *8*(1), 32–41. https://doi.org/10.1109/52.62930

Gibbs, G. (1988). *Learning by doing: A guide to teaching and learning methods*. Further Education Unit, Oxford Polytechnic.

Hersey, P., & Blanchard, K. H. (1969). Life cycle theory of leadership. *Training and Development Journal*, *23*(5), 26–34.

Johnson, D. W., & Johnson, R. T. (2009). An educational psychology success story: Social interdependence theory and cooperative learning. *Educational Researcher*, *38*(5), 365–379. https://doi.org/10.3102/0013189X09339057

Kolb, D. A. (1984). *Experiential learning: Experience as the source of learning and development*. Prentice Hall.

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, *2*, 79. https://doi.org/10.22331/q-2018-08-06-79

Schön, D. A. (1983). *The reflective practitioner: How professionals think in action*. Basic Books.

Tuckman, B. W. (1965). Developmental sequence in small groups. *Psychological Bulletin*, *63*(6), 384–399. https://doi.org/10.1037/h0022100

Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P., & Wilson, P. (2014). Best practices for scientific computing. *PLoS Biology*, *12*(1), Article e1001745. https://doi.org/10.1371/journal.pbio.1001745

Appendix – Evidence Behind This Reflection

- Git history: https://github.com/anishnz/quantumcomputing_YB_Anish (commits from 31 October and 4 November 2026).
- Quokka test evidence: `Device_Comparison/results/quokka_status_2026-11-04.json` and the dated screenshots in `Device_Comparison/screenshots/`.
- Laptop vs Quokka data: `Device_Comparison/results/comparison_summary.csv`.
- Task 2 redesign: `Task_2_Qiskit_QASM/results/task2a_run1.json` to `task2a_run3.json` (source QUOKKA DEVICE, quokka2).
- Task 3 played games: `Task_3_Quantum_TicTacToe/results/gameplay/` (screenshots and log).